Post-Quantum Cryptography Web Application

Sarim Aeyzaz (i21-0328) Ehtsham Walidad (i21-0260)

GitHub Repository

Link to project repository: `github.com/Yusixs/post-quantum-cryptography`

1. Overview

This project demonstrates a quantum-safe encryption and decryption web application using a Post-Quantum Cryptography (PQC) algorithm. Although the assignment specified Flask, we implemented the backend using **FastAPI** for modern async support, better developer experience, and performance.

The application performs key generation, message encryption using KEM + AES hybrid encryption, and decryption via the corresponding secret key. A user-friendly front-end interface has been integrated using HTML, CSS, and JavaScript.

2. Tools and Technologies

- **Programming Language:** Python 3.11
- **Backend Framework:** FastAPI (instead of Flask)
- **Cryptography:** pqcrypto.kem (ML-KEM-512)
- **Frontend:** HTML, CSS (Bootstrap optional)
- **Others:** Jinja2, base64, logging, AES (via `cryptography` package)

3. PQC Algorithm Used

Our application uses **ML-KEM-512**, a post-quantum Key Encapsulation Mechanism (KEM) based on the **Kyber** algorithm. It is one of the standardized algorithms selected by **NIST** for quantum-resistant public-key encryption and key exchange.

What is ML-KEM?

ML-KEM stands for **Module-Lattice-based Key Encapsulation Mechanism**. It is a cryptographic protocol that enables two parties (like Alice and Bob) to agree on a shared secret over an insecure channel — even if powerful quantum computers are eavesdropping. To visualize it, imagine:

- Alice locks a box (the ciphertext) with a padlock (the public key) and sends it to Bob.
- Only Bob, who has the key (the secret key), can unlock the box to retrieve the shared secret inside.

Unlike RSA or ECC (which would break under quantum attacks), ML-KEM relies on problems from lattice mathematics — specifically the Module Learning With Errors (MLWE) problem — which are believed to be hard even for quantum computers.

Why ML-KEM-512?

ML-KEM-512 is the smallest and fastest variant among the ML-KEM (Kyber) family:

- It provides a classical security level of 128 bits (comparable to AES-128).
- It has smaller key and ciphertext sizes compared to stronger variants like ML-KEM-768 or ML-KEM-1024.
- Ideal for performance-sensitive applications like web apps, IoT devices, and mobile environments.

How It Works — In Depth

The process consists of three main operations:

1. **Key Generation:** Generates a public and secret key.
    - Think of this as Bob creating a safe (public key) and keeping the combination (secret key) secret.
2. **Encapsulation:** Alice uses the public key to generate a shared secret and a ciphertext.
    - It's like Alice placing the shared secret into Bob's safe and sending it back to him.
3. **Decapsulation:** Bob uses the secret key to recover the shared secret from the ciphertext.
    - He uses the secret combination to open the safe and retrieve the secret inside.

Mathematically, ML-KEM relies on noisy polynomial arithmetic in modular rings, but the noise is intentional — it prevents attackers from reversing the process even if they intercept the ciphertext.

Security Assumptions

ML-KEM is based on the **Module Learning With Errors (MLWE)** problem. This problem involves solving noisy linear equations in a high-dimensional lattice — a task that's hard for both classical and quantum computers.

In simple terms:

- Imagine shouting your phone number across a noisy room (this is the ciphertext).
- Your friend, who knows the structure of the noise (the secret key), can decode what you said.
- An eavesdropper, hearing only the noise and muffled digits, can't reconstruct your message.

Why Lattices?

Lattice-based cryptography is appealing because:

- It resists known quantum attacks (unlike RSA or ECC).
- It allows relatively efficient operations (additions, multiplications, noise injection).
- It supports small key sizes and fast computation, especially in Kyber/ML-KEM.

Hybrid Encryption Design in Our App

ML-KEM-512 handles only the **key exchange**, not message encryption. That's why we use a **hybrid scheme**:

- ML-KEM encapsulates a shared secret.
- AES-CBC uses that secret to encrypt the actual message.

This mirrors the TLS (HTTPS) model where RSA/DH/ECDH exchange a key, and symmetric encryption handles the bulk data.

4. Application Functionality

Key Generation

- Generates a public/secret key pair.
- Keys are base64 encoded for UI display and transmission.

Message Encryption

- Shared secret is encapsulated using the public key (KEM).
- AES-CBC is used to encrypt the message with the shared secret.

Message Decryption

- Secret key is used to decapsulate the shared secret.
- AES-CBC is used to decrypt the message using that secret.

5. Implementation Challenges and Deployment

FastAPI Instead of Flask

Although the assignment required Flask, FastAPI was used as a drop-in modern replacement. Routing, templating, and security middleware were adapted accordingly.

pqcrypto Integration

The `pqcrypto` library required manual setup:

- We had to clone the `pqcrypto` Git repository and its submodules.
- After compiling the submodules, the `_kem` and `_sign` folders were copied into the `pqcrypto` package.

Deployment via Docker

Despite the library's incompatibility with standard Python package managers like `pip`, we were able to successfully deploy the application on **Render** using a `Dockerfile`.

- The Docker image includes a step to compile the `pqcrypto` library inside the container.
- This ensured that the FastAPI server and cryptographic modules were fully operational in the deployment environment.
- The app can now be accessed live via a public Render URL.

This setup bypassed the limitations of `pip install -r requirements.txt` and enabled a portable, reproducible deployment environment using containerization.

6. Screenshots

Note: Ensure the above screenshots are available in the `images/` folder before compiling.

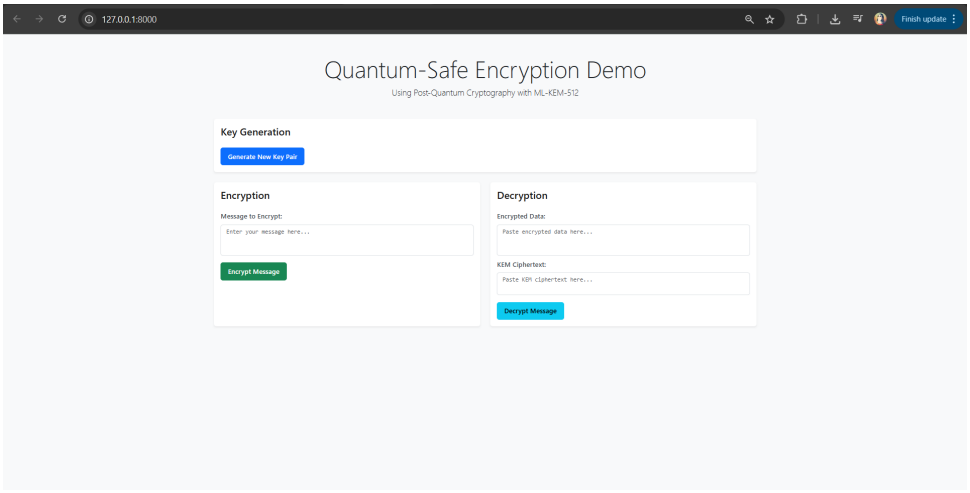

Figure 1: Homepage before key generation

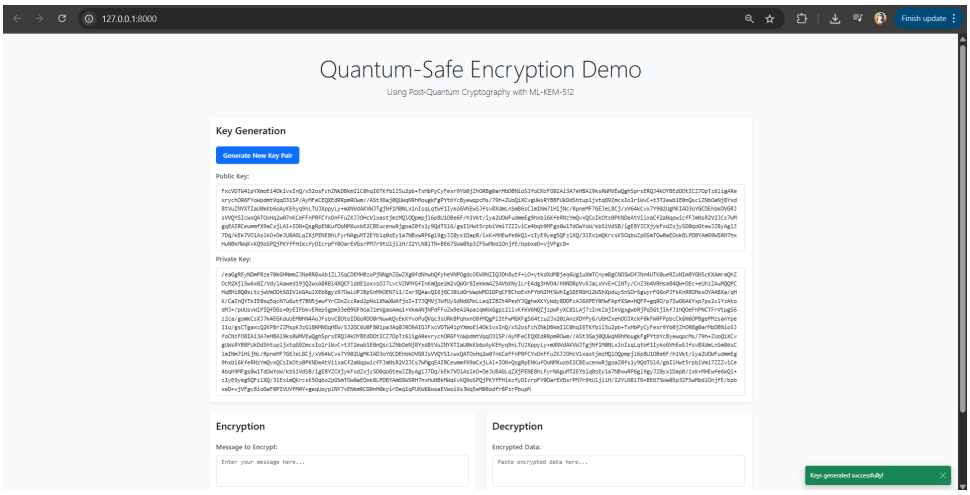

Figure 2: Public/Secret key after generation

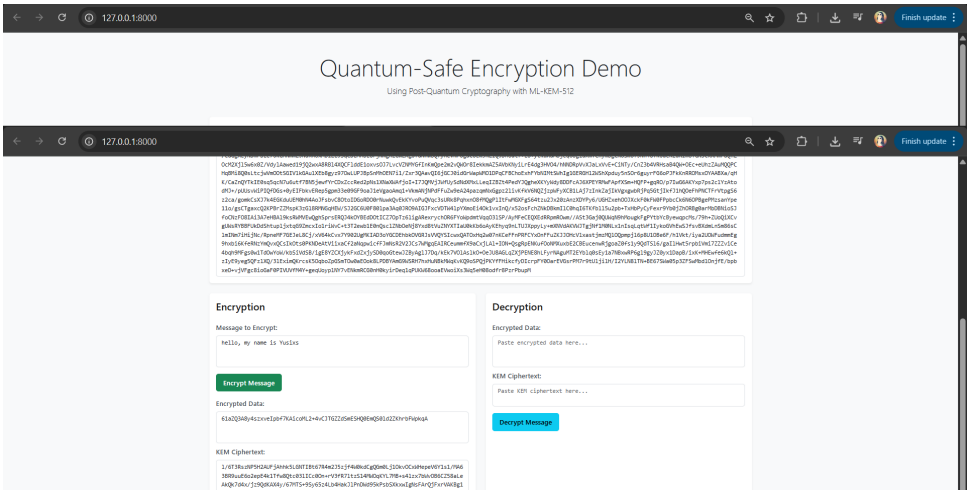

Figure 3: Encryption interface

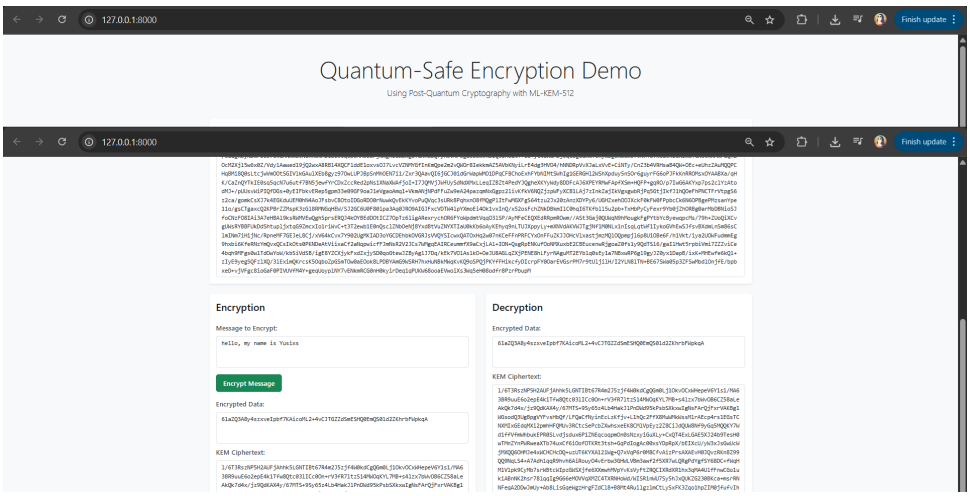

Figure 4: Decryption result

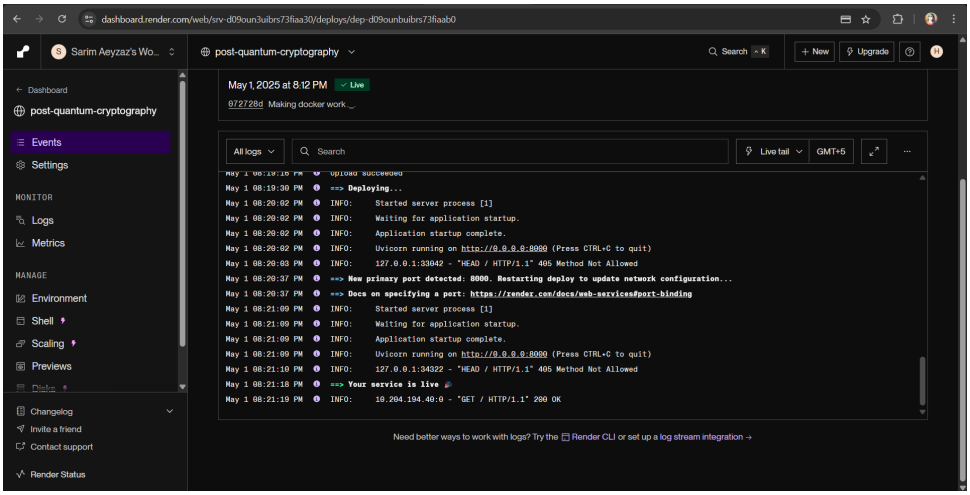

Figure 5: Deployed application on Render

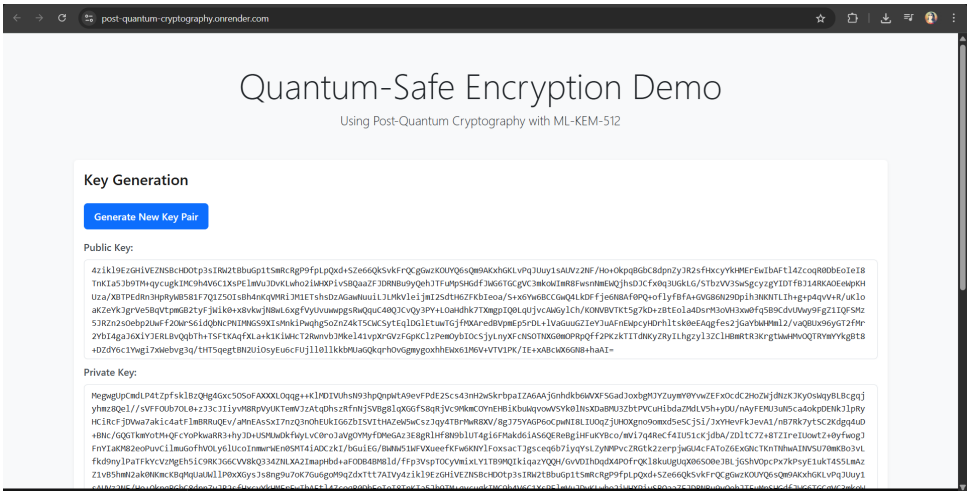

Figure 6: Deployed application on Render – key generation in action